Great Kartik 👍
Now we are starting the **Database Phase**, which is essential for MERN placement.

In MERN stack, the database used is **MongoDB** and we connect it with **Node.js** using **Mongoose**.

🚀 Lesson 24 — Introduction to MongoDB

1 What is MongoDB?

MongoDB is a **NoSQL database** used to store application data.

Unlike SQL databases, it stores data in **JSON-like documents**.

Example data:

```
{
  "name": "Kartik",
  "email": "kartik@gmail.com",
  "age": 22
}
```

This format is called **BSON (Binary JSON)**.

---

2 Why MongoDB is Used in MERN

Reasons developers use **MongoDB**:

✓ Works naturally with JavaScript
✓ Flexible schema
✓ High performance
✓ Easy to scale

This is why it is used with **React**, **Express.js**, and **Node.js**.

---

3 MongoDB Data Structure

MongoDB has 3 main levels:

Database

↓

Collection

↓

Document

Example:

Database → companyDB

Collection → users

Document → {name:"Kartik", age:22}

Example documents:

```
[
 { "name": "Kartik", "age": 22 },
 { "name": "Rahul", "age": 24 }
]
```

---

4 SQL vs MongoDB

SQL	MongoDB
Table	Collection
Row	Document
Column	Field

Example:

SQL:

users table

MongoDB:

users collection

---

5 Tools for MongoDB

Developers usually install:

MongoDB Server

Database engine.

MongoDB Compass

GUI tool to **view and manage database visually**.

Example:

Create database

Insert documents

View collections

---

6 Example Document

User document:

```
{
 "_id": "12345",
 "name": "Kartik",
 "email": "kartik@gmail.com",
 "password": "hashedpassword"
}
```

MongoDB automatically creates:

_id

This is the **unique identifier**.

🧠 **Interview Question**

What is MongoDB?

Answer:

MongoDB is a document-based NoSQL database that stores data in flexible JSON-like documents.

📊 **Course Progress**

[redacted]

≈ 96% Backend Course Completed

You finished:

✓ Node.js fundamentals
✓ Express backend
✓ MVC architecture
✓ Validation
✓ Authentication
✓ Authorization
✓ Environment variables

Now we are in **Database phase**.

⏭ **Next Lesson (Very Important)**

Next we will learn:

MongoDB Installation + MongoDB Compass Setup

Then we will start **real database operations**.

After that:

Connecting Node.js to MongoDB

CRUD operations

Mongoose schemas

Models

Type **"next lesson"** and we continue 🚀.

Perfect Kartik ✅
Your installation is **successful**. I can see that **MongoDB Compass** is connected to **MongoDB** at:

localhost:27017

The databases you see:

admin

config

local

are **default system databases**, so everything is working correctly. 👍

🚀 **Now Let's Do Your First Database Step**

1 Create Your Own Database

On the left side click:

+ (Create Database)

Then fill:

Database Name: mernDB

Collection Name: users

Click:

Create Database

Now structure will look like:

```
mernDB
  |
  |-- users
```

---

2 Insert First Document

Open **users collection** and click:

Insert Document

Add this:

```
{
 "name": "Kartik",
 "email": "kartik@gmail.com",
 "age": 22
}
```

Click **Insert**.

Now MongoDB will automatically add:

_id

Example:

```
{
 "_id": "665d9a...",
 "name": "Kartik",
 "email": "kartik@gmail.com",
 "age": 22
}
```

🗄 What You Just Learned

You now know:

✓ How to install **MongoDB**
✓ How to connect with **MongoDB Compass**
✓ How to create database
✓ How to insert documents

🚀 Next Lesson (Very Important)

Now we will connect your backend:

Node.js → MongoDB

using **Mongoose**.

This is where **real MERN backend coding starts**.

👉 If you want, I can also show you **the exact backend folder structure companies use for Node + MongoDB projects** before we connect the database.

Great Kartik 👍
Now we start **real database coding** by connecting **Node.js** with **MongoDB** using **Mongoose**.

This is how **most MERN backends connect to the database**.

🚀 Lesson 26 — Connect Node.js to MongoDB

1 Install Mongoose

Open your backend project terminal and run:

npm install mongoose

This installs **Mongoose**.

---

2 Import Mongoose

Inside your **server.js**:

```
const mongoose = require("mongoose");
```

---

3 Connect to MongoDB

Add this code:

```
const mongoose = require("mongoose");
mongoose.connect("mongodb://127.0.0.1:27017/mernDB")
.then(()=>{
  console.log("MongoDB Connected");
})
.catch((error)=>{
  console.log(error);
});
```

Explanation:

Part	Meaning
mongodb://	MongoDB protocol
127.0.0.1	your computer
27017	default MongoDB port
mernDB	your database

---

4 Full Example Server

```
const express = require("express");
const mongoose = require("mongoose");
```

```
const app = express();
mongoose.connect("mongodb://127.0.0.1:27017/mernDB")
.then(()=>{
 console.log("MongoDB Connected");
})
.catch(err=>console.log(err));
app.listen(3000,()=>{
 console.log("Server running");
});
```

---

5 Run Server

In terminal run:

```
node server.js
```

Output should be:

```
MongoDB Connected
Server running
```

This means:

Node.js server → connected to MongoDB

🌐 How MERN Flow Works Now

React (Frontend)

↓

Express API

↓

Mongoose

↓

MongoDB Database

📊 **Course Progress**

[redacted]

≈ 97% Completed

You now have:

✓ Backend server
✓ Authentication system
✓ MongoDB database
✓ Database connection

⏭ **Next Lesson (Very Important)**

Next we will learn:

Mongoose Schema

This defines the **structure of documents in MongoDB**.

Example:

name: String

email: String

password: String

Schemas are **required before creating models and CRUD APIs**.

Type **"next lesson"** and we continue 🚀.

Great Kartik 👍
Now we move to a **very important concept** before CRUD operations.

This is **Mongoose Schema**, used with **Mongoose** to define the structure of data stored in **MongoDB**.

🚀 **Lesson 27 — Mongoose Schema**

1 What is a Schema?

A **Schema** defines the **structure of documents** inside a collection.

Without schema:

{

```
 "name": "Kartik",
 "age": 22
}
```

Someone could also insert:

```
{
 "random": "data",
 "wrong": 123
}
```

❌ No control on data

With Schema:

name → String

email → String

age → Number

✓ Data is controlled and structured

---

2 Create a Schema

Create file:

models/userModel.js

Code:

```
const mongoose = require("mongoose");
const userSchema = new mongoose.Schema({
 name: String,
 email: String,
 age: Number
});

module.exports = userSchema;
```

---

3 Schema with Validation

We can also add rules:

```
const mongoose = require("mongoose");

const userSchema = new mongoose.Schema({

 name: {

  type: String,

  required: true

 },

 email: {

  type: String,

  required: true,

  unique: true

 },

 age: {

  type: Number,

  min: 18

 }

})

module.exports = userSchema;
```

---

4 Schema Features

You can control:

✓ data type
✓ required fields
✓ uniqueness
✓ min/max values
✓ default values

---

5 Example Document (After Schema)

```
{
 "_id": "12345",
 "name": "Kartik",
 "email": "kartik@gmail.com",
 "age": 22
}
```

If someone tries:

```
{
 "name": "",
 "age": 10
}
```

❌ It will fail validation

🧠 **Interview Question**

What is a Mongoose Schema?

Answer:

A schema defines the **structure, data types, and validation rules** for documents in a MongoDB collection using **Mongoose**.

📊 **Course Progress**

[redacted]

≈ 98% Completed

You are now at **final stage of backend mastery**.

⏭ **Next Lesson (Very Important)**

Next we will learn:

Mongoose Models

This is where schema becomes usable.

Flow:

Schema → Model → Database Operations

After that:

🔥 Real CRUD APIs with database

Type **“next lesson”** 🚀

Got you 👍 Kartik — I’ll explain **Mongoose Model** again in a very simple way (no confusion).

🧠 First Understand the Flow

Schema → Model → Database

- **Schema** = rules (structure of data)
- **Model** = tool to use those rules
- **MongoDB** = where data is stored

🔥 What is a Model? (Simple Meaning)

A **Model** is like a **bridge** between your backend (**Node.js**) and database (**MongoDB**).

👉 It lets you:

- Insert data
- Read data
- Update data
- Delete data

Using **Mongoose**.

📦 Real Life Example

Think:

- Schema = Form design (Name, Email, Age)

- Model = Person who fills & submits form
- MongoDB = Storage room

📄 Code Step-by-Step

1 Schema (rules)

```
const mongoose = require("mongoose")
const userSchema = new mongoose.Schema({
 name: String,
 email: String,
 age: Number
});
```

---

2 Model (important line)

```
const User = mongoose.model("User", userSchema);
```

👉 Meaning:

- "User" → collection name (MongoDB creates **users**)
- userSchema → structure

---

3 Export Model

```
module.exports = User;
```

🚀 Now Use Model in Server

```
const User = require("./models/userModel");
```

🔥 Insert Data (Most Important Part)

```
app.post("/add-user", async (req, res) => {
 const user = new User(req.body); // create object
```

```
 await user.save(); // save to database


 res.send("User Saved");
});
```

⚡ What Happens Internally

When you send request:

```
{
 "name": "Kartik",
 "email": "kartik@gmail.com",
 "age": 22
}
```

Flow:

Frontend → Express → Model → MongoDB

💡 Key Line (Remember This)

```
const user = new User(req.body);
```

👉 This creates a new document

```
await user.save();
```

👉 This stores data in MongoDB

🎯 Super Short Summary

Schema = structure

Model = used to talk to DB

save() = store data

🧪 Mini Practice (Do This Now)

Create 2 users using Postman:

```
{
 "name": "Rahul",
 "email": "rahul@gmail.com",
 "age": 25
}
{
 "name": "Aman",
 "email": "aman@gmail.com",
 "age": 21
}
```

Check in **MongoDB Compass** → data will appear ✅

🚀 **Next Step**

Now you are ready for **CRUD operations (real backend skill)**:

- Read data
- Update data
- Delete data

Type **"next lesson"** and we finish your backend 🔥

Perfect Kartik 🔥
Now we reach the **most important practical part** — this is what companies actually test.

🚀 **Lesson 29 — CRUD Operations with Database**

We will use **Mongoose** with **MongoDB**.

CRUD =

Create ✅ (you already did)

Read ✅ (now)

Update ✅

Delete ✅

🧠 1 READ (Get Data)

Get all users

```
app.get("/users", async (req, res) => {
 const users = await User.find();
 res.json(users);
});
```

👉 This returns all users from database

Get single user

```
app.get("/users/:id", async (req, res) => {
 const user = await User.findById(req.params.id);
 res.json(user);
});
```

🧠 2 UPDATE Data

```
app.put("/users/:id", async (req, res) => {
 const updatedUser = await User.findByIdAndUpdate(
  req.params.id,
  req.body,
  { new: true }
 );
 res.json(updatedUser);
});
```

👉 { new: true } → returns updated data

🧠 3 DELETE Data

```
app.delete("/users/:id", async (req, res) => {
 await User.findByIdAndDelete(req.params.id);
 res.send("User Deleted");
});
```

🔁 Full CRUD Summary

Operation	Method	API
Create	POST	/add-user
Read All	GET	/users
Read One	GET	/users/:id
Update	PUT	/users/:id
Delete	DELETE	/users/:id

🔥 Complete Example

```
app.post("/add-user", async (req, res) => {
 const user = new User(req.body);
 await user.save();
 res.send("User Saved");
});
app.get("/users", async (req, res) => {
 const users = await User.find();
 res.json(users);
});


app.put("/users/:id", async (req, res) => {
```

```
 const updatedUser = await User.findByIdAndUpdate(
  req.params.id,
  req.body,
  { new: true }
 );
 res.json(updatedUser);
});
app.delete("/users/:id", async (req, res) => {
 await User.findByIdAndDelete(req.params.id);
 res.send("User Deleted");
});
```

🔄 **Flow of CRUD**

Client (Postman / React)

↓

Express API

↓

Mongoose Model

↓

MongoDB Database

🧪 **Practice (Very Important)**

Test all APIs in Postman:

1 Create

POST /add-user

2 Read

GET /users

3 Update

PUT /users/:id

4 Delete

DELETE /users/:id

🧠 **Interview Questions**

Q1: What is CRUD?

👉 Create, Read, Update, Delete operations on database

Q2: How to fetch data in MongoDB?

👉 Model.find()

📊 **Course Progress**

≈ 100% Core Backend Completed 🎉

🚀 **What's Next (Important for Placement)**

Even though core is done, **placement-level topics still remain**:

- Query Filters
- MongoDB Operators
- Relationships & Populate
- Aggregation
- File Upload (Multer)
- Security (Helmet, CORS)
- Deployment

💡 Now you are **officially able to build full backend APIs**.

👉 Next lesson will be **Advanced Queries (Filters + Operators)** — very important for real projects.

Type **“next lesson”** 🚀

No problem Kartik 👍 — I’ll explain **Query Filters & Operators** again in a very simple + practical way.

🤔 What is Filtering?

👉 Filtering means: **get only specific data from database**

Example:

You have 100 users
But you want only users with **age = 22**

That is called **filtering**

🔥 Basic Syntax

Using **Mongoose** with **MongoDB**:

User.find({ condition })

>1 Simple Filter

👉 Get users with age = 22

User.find({ age: 22 });

🚀 2 Operators (Most Important 🔥)

Operators help you apply conditions.

🔷 Greater Than ($gt)

User.find({ age: { $gt: 20 } });

👉 age > 20

🔷 Less Than ($lt)

User.find({ age: { $lt: 25 } });

👉 age < 25

◈ Range

User.find({ age: { $gt: 20, $lt: 30 } });

👉 20 < age < 30

◈ Not Equal ($ne)

User.find({ age: { $ne: 22 } });

👉 NOT 22

◈ Multiple Values ($in)

User.find({ age: { $in: [21, 22, 23] } });

👉 age = 21 OR 22 OR 23

🎯 Easy Way to Remember

$gt → >

$lt → <

$ne → ≠

$in → OR

🌐 3 Real API Example (Very Important)

Instead of hardcoding:

User.find({ age: 22 });

We take input from URL:

```
app.get("/users", async (req, res) => {
 const age = req.query.age;
```

```
 const users = await User.find({ age: age });
 res.json(users);
});
```

📎 How You Call API

http://localhost:3000/users?age=22

👉 Now API becomes **dynamic**

🔍 4 Search (Very Important 🔥)

Search by name:

```
User.find({
 name: { $regex: "kar", $options: "i" }
});
```

👉 finds:

- Kartik
- Karan
- KARAN

🧠 Simple Understanding

Filter = choose data

Operator = condition

Query = user input from URL

🧪 Real Life Example

Think Instagram:

- Search user → $regex
- Filter age → $gt, $lt

- Select multiple → $in

⚡ Mini Practice

Try these in your code:

User.find({ age: { $gt: 21 } });

User.find({ age: { $lt: 25 } });

User.find({ name: { $regex: "ka", $options: "i" } });

🚀 Next Step

Now next topic is:

👉 **Sorting + Limit + Pagination**
(this is used in every app like YouTube, Instagram)

Type **"next lesson"** 🚀

Good 👍 Kartik — let's break this **line by line** so you fully understand it.

🔍 Your Code

```
app.get("/users", async (req, res) => {
 const age = req.query.age;
 const users = await User.find({ age: age });
 res.json(users);
});
```

🧠 Step-by-Step Explanation

---

1 app.get("/users", ...)

👉 This creates an **API route**

GET /users

Means:

When you open:

http://localhost:3000/users

this function will run.

---

2 req.query.age

👉 This gets data from **URL query**

Example URL:

http://localhost:3000/users?age=22

Now:

const age = req.query.age;

👉 age = "22" (comes as string)

---

3 User.find({ age: age })

Using **Mongoose** to query **MongoDB**

👉 This means:

Find users where age = 22

⚠️ Important:

MongoDB stores age as **Number**, but req.query.age is **String**

So better way:

const users = await User.find({ age: Number(age) });

---

4 res.json(users)

👉 Sends data back as JSON

Example response:

```
[
 {
```

```
  "_id": "123",
  "name": "Kartik",
  "age": 22
 }
]
```

🔁 **Full Flow**

```
Browser/Postman
    ↓
GET /users?age=22
    ↓
req.query.age → 22
    ↓
User.find({ age: 22 })
    ↓
MongoDB
    ↓
Response sent (JSON)
```

💡 **Final Improved Code (Best Practice)**

```
app.get("/users", async (req, res) => {
 try {
  const age = Number(req.query.age);
  const users = await User.find({ age });
  res.json(users);
 } catch (err) {
  res.status(500).send(err.message);
 }
```

```
});
```

🚀 **Super Important Understanding**

req.query → get data from URL

find() → fetch from DB

res.json() → send response

🧪 **Try This**

Open browser:

http://localhost:3000/users?age=22

👉 You will get only users with age = 22

If you want, next I can show:

👉 multiple filters (age + name together)
👉 or sorting & pagination (very important for placement)

Just say **next lesson** 🚀

Perfect Kartik 🔥 — now we move to a **VERY IMPORTANT topic for placement**.

This is used in **every real app** like Instagram, YouTube, Amazon.

🚀 **Lesson 31 — Sorting, Limit & Pagination**

Using **Mongoose** with **MongoDB**

🧠 **1 Sorting Data**

👉 Sort users by age

```
User.find().sort({ age: 1 });
```

Meaning:

1 → Ascending (small → big)

-1 → Descending (big → small)

◈ Example

User.find().sort({ age: -1 });

👉 Oldest users first

🧠 2 Limit (Control Data)

👉 Get only 2 users

User.find().limit(2);

◈ Real Use

- Show top 5 users
- Show latest 10 posts

🧠 3 Pagination (VERY IMPORTANT 🔥)

👉 Used when data is large

Example:

- Page 1 → first 5 users
- Page 2 → next 5 users

◈ Formula

skip = (page - 1) * limit

◈ Code Example

```
app.get("/users", async (req, res) => {
 const page = Number(req.query.page) || 1;
 const limit = Number(req.query.limit) || 5;
```

```
 const users = await User.find()
  .skip((page - 1) * limit)
  .limit(limit);
 res.json(users);
});
```

◈ API Call

http://localhost:3000/users?page=1&limit=5

🧠 Combine Everything (🔥 Real API)

```
app.get("/users", async (req, res) => {
 const page = Number(req.query.page) || 1;
 const limit = Number(req.query.limit) || 5;
 const sort = req.query.sort || "age";
 const users = await User.find()
  .sort({ [sort]: 1 })
  .skip((page - 1) * limit)
  .limit(limit);
 res.json(users);
});
```

🔲 Full Flow

Request → Query Params → DB Query → Filtered Result

🧪 Practice

Try these:

1 Page 1

/users?page=1&limit=2

2 Page 2

/users?page=2&limit=2

3 Sorting

/users?sort=age

🎯 **Interview Questions**

Q1: What is pagination?

👉 Dividing data into pages

Q2: Why use pagination?

👉 To handle large data efficiently

Q3: What is skip()?

👉 Skips records

📊 **Progress**

[redacted]

≈ 94% Completed

🚀 **Next Lesson**

Next we learn:

👉 **Relationships (Populate)** 🔥
(Used like foreign key in SQL)

Example:

User → Posts → Comments

Type **"next lesson"** 🚀

Good question Kartik 👍 — this is exactly how real apps work 🔥

You want:

👉 **Latest users + top 2 only**

🧠 **Key Idea**

Sort first → then limit

> Code

```
const users = await User.find()
  .sort({ createdAt: -1 }) // latest first
  .limit(2);               // take only 2
```

🔍 **Explanation**

🔹 **.sort({ createdAt: -1 })**

- -1 → descending
- newest users come first

🔹 **.limit(2)**

- only return **top 2 users**

⚠️ **Important Condition**

👉 This works only if you have **createdAt field**

If not, add timestamps in schema:

```
const userSchema = new mongoose.Schema({
 name: String,
 email: String,
 age: Number
}, { timestamps: true });
```

Now MongoDB will automatically create:

```
{
 "createdAt": "2026-03-17T...",
```

```
 "updatedAt": "2026-03-17T..."
}
```

🚀 Full API Example

```
app.get("/latest-users", async (req, res) => {
 const users = await User.find()
  .sort({ createdAt: -1 })
  .limit(2);
 res.json(users);
});
```

🧠 Real-Life Example

Same logic used in:

- Instagram → latest posts
- YouTube → latest videos
- Amazon → newest products

🎯 Interview Tip

If asked:

👉 *"How to get latest records?"*

Say:

Use .sort({ createdAt: -1 }).limit(n) in MongoDB.

💡 Shortcut Rule

Latest = sort(-1)

Top N = limit(n)

If you want next level:

👉 combine **filter + sort + pagination (advanced API)**
👉 OR move to **relationships (populate)**

Just say **next lesson** 🚀

Perfect Kartik 🔥 — now we enter a **very important real-world concept**:

🚀 Lesson 32 — Relationships (Populate)

This is like **foreign key in SQL**, but in **MongoDB** we use **populate** from **Mongoose**.

🌍 What is Relationship?

👉 One collection connected to another

Example:

User → Posts

- One user can have many posts
- Each post belongs to one user

🔥 1 Create Post Schema (Reference)

Create new file:

models/postModel.js

```
const mongoose = require("mongoose");

const postSchema = new mongoose.Schema({

 title: String,

 content: String,

 user: {

  type: mongoose.Schema.Types.ObjectId,

  ref: "User"

 }

});
```

```
const Post = mongoose.model("Post", postSchema);

module.exports = Post;
```

🧠 **Important Line**

ref: "User"

👉 This connects **Post → User**

🚀 **2 Create Post API**

```
const Post = require("./models/postModel");

app.post("/add-post", async (req, res) => {
 const post = new Post(req.body);
 await post.save();
 res.send("Post Saved");
});
```

🧪 **Example Data**

```
{
 "title": "My First Post",
 "content": "Hello World",
 "user": "USER_ID_HERE"
}
```

👉 Put **user _id** from MongoDB

🚀 **3 Populate (MOST IMPORTANT 🔥)**

👉 Fetch post with user details

```
app.get("/posts", async (req, res) => {
 const posts = await Post.find().populate("user");
```

```
 res.json(posts);
});
```

🔍 **What Happens?**

Without populate:

```
{
 "title": "Post",
 "user": "65abcd123"
}
```

With populate:

```
{
 "title": "Post",
 "user": {
  "_id": "65abcd123",
  "name": "Kartik",
  "email": "kartik@gmail.com"
 }
}
```

🎯 **Why Use Populate?**

👉 To join data from different collections

Same as:

SQL → JOIN

MongoDB → populate()

🧠 **Advanced Populate (Optional)**

```
Post.find().populate("user", "name email");
```

👉 Only return name & email

🔁 **Flow**

Post → userId → populate → full user data

🧪 **Practice**

1 Create user
2 Copy _id
3 Create post with that id
4 Call /posts

🎯 **Interview Questions**

Q1: What is populate?

👉 Used to fetch referenced data from another collection

Q2: How to create relationship in MongoDB?

👉 Using ObjectId and ref

📊 **Progress**

[redacted]

≈ 96% Completed

🚀 **Next Lesson**

Now only powerful topics left:

👉 Aggregation (basic)
👉 File Upload (Multer)
👉 Security

Type **"next lesson"** 🚀

Perfect Kartik 🔥 — now we move to a **powerful topic used in real projects + interviews**.

🚀 Lesson 33 — Aggregation (Basic)

Aggregation in **MongoDB** is used to:

Analyze + Transform Data

Using **Mongoose**

🧠 What is Aggregation?

👉 It processes data in **steps (pipeline)**

Example:

Data → Filter → Group → Result

🔥 1 Basic Syntax

```
User.aggregate([
  { stage1 },
  { stage2 }
]);
```

🧠 2 Match (Filter Data)

Same as find()

```
User.aggregate([
  { $match: { age: { $gt: 20 } } }
]);
```

👉 age > 20

🧠 3 Group (Very Important 🔥)

👉 Group data and calculate values

```
User.aggregate([
 {
```

```
  $group: {
   _id: "$age",
   totalUsers: { $sum: 1 }
  }
 }
]);
```

🔍 **Output**

```
[
 { "_id": 22, "totalUsers": 3 },
 { "_id": 25, "totalUsers": 2 }
]
```

👉 Means:

- 3 users with age 22
- 2 users with age 25

🧠 **4 Sort in Aggregation**

```
User.aggregate([
 { $sort: { age: -1 } }
]);
```

🧠 **5 Limit**

```
User.aggregate([
 { $limit: 2 }
]);
```

🚀 **6 Real Example (🔥 Important)**

👉 Count users by age > 20

```
app.get("/stats", async (req, res) => {
 const data = await User.aggregate([
  { $match: { age: { $gt: 20 } } },
  {
   $group: {
    _id: "$age",
    count: { $sum: 1 }
   }
  },
  { $sort: { count: -1 } }
 ]);
 res.json(data);
});
```

🔁 **Flow**

Collection → Match → Group → Sort → Result

🧠 **Easy Understanding**

Stage	Meaning
$match	filter
$group	combine
$sort	order
$limit	restrict

🎯 **Real Use Cases**

- Dashboard stats

- Analytics
- Reports
- Admin panels

🧪 Practice

Try:

```
User.aggregate([
 { $group: { _id: "$age", count: { $sum: 1 } } }
]);
```

🎯 Interview Questions

Q1: What is aggregation?

👉 Data processing pipeline

Q2: Difference between find and aggregate?

👉 find = simple query
👉 aggregate = complex data processing

📊 Progress

[redacted]

≈ 97% Completed

🚀 Next Lesson

Now only 2 powerful topics left:

👉 File Upload (Multer) 🔥
👉 Security (CORS, Helmet)

Type **"next lesson"** 🚀

Perfect Kartik 🔥 — now we learn a **very practical + project-level topic**:

🚀 Lesson 34 — File Upload (Multer)

This is used in real apps for:

Profile Image Upload

PDF Upload

File Storage Systems

We use **Multer** with **Express**.

🧠 1 Install Multer

```
npm install multer
```

🧠 2 import Multer

```
const multer = require("multer");
```

🧠 3 Storage Configuration

```
const storage = multer.diskStorage({
 destination: function (req, file, cb) {
  cb(null, "uploads/");
 },
 filename: function (req, file, cb) {
  cb(null, Date.now() + "-" + file.originalname);
 }
});
```

🧠 4 Create Upload Middleware

```
const upload = multer({ storage: storage });
```

🚀 5 Create Upload API

```
app.post("/upload", upload.single("file"), (req, res) => {
 res.send("File Uploaded");
});
```

🔍 Important Line

```
upload.single("file")
```

👉 "file" must match **form field name**

🧪 How to Test (Postman)

1 Method → POST
2 URL → /upload

3 Body → form-data
4 Key → file
5 Type → File
6 Choose file

📂 Folder Structure

```
project/
  ├── uploads/
  ├── server.js
```

👉 Uploaded files will go inside **uploads folder**

🌐 6 Access Uploaded File

Example file path:

uploads/171000-file.png

🚀 Serve Files (Important)

```
app.use("/uploads", express.static("uploads"));
```

Now access:

http://localhost:3000/uploads/file.png

🔥 Advanced (Multiple Files)

```
app.post("/upload-multiple", upload.array("files", 5), (req, res) => {
 res.send("Multiple files uploaded");
});
```

🎯 Real Use Cases

- Profile image upload
- Resume upload
- Secure file storage system (your project 🔥)

🧠 Interview Questions

Q1: What is Multer?

👉 Middleware for handling file uploads

Q2: What is diskStorage?

👉 Controls where & how file is stored

📊 Progress

≈ 99% Completed

🚀 Final Lesson Left

👉 Security (CORS, Helmet, Rate Limit) 🔥

This is **very important for placement + real apps**

Type **“next lesson”** 🚀

Perfect Kartik 🔥 — this is the **final backend lesson (very important for placement)**.

🚀 Lesson 35 — Security (CORS, Helmet, Rate Limiting)

In real-world apps, security is **mandatory**.

We use these tools:

- **cors**
- **helmet**
- **express-rate-limit**

🌐 1 CORS (Cross-Origin Resource Sharing)

👉 Allows frontend (React) to call backend

❌ Problem

Without CORS:

Frontend (localhost:5173) ❌ Backend (localhost:3000)

Browser blocks request

✅ Solution

Install:

```
npm install cors
```

Use:

```
const cors = require("cors");


app.use(cors());
```

🔒 **Restrict Access (Advanced)**

```
app.use(cors({
 origin: "http://localhost:5173"
}));
```

👉 Only your frontend can access API

🧠 **2 Helmet (Security Headers)**

👉 Protects app from common attacks

Install:

```
npm install helmet
```

Use:

```
const helmet = require("helmet");
app.use(helmet());
```

🧠 **3 Rate Limiting (Prevent Attacks)**

👉 Limits number of requests

Install

```
npm install express-rate-limit
```

Use

```
const rateLimit = require("express-rate-limit");
const limiter = rateLimit({
 windowMs: 15 * 60 * 1000, // 15 minutes
 max: 100
});
app.use(limiter);
```

🔥 What This Does

User can make only 100 requests in 15 minutes

👉 Prevents:

- brute force attacks
- API spam
- server crash

🚀 Full Secure Setup

```
const express = require("express");

const cors = require("cors");

const helmet = require("helmet");

const rateLimit = require("express-rate-limit");

const app = express();

app.use(cors());

app.use(helmet());

const limiter = rateLimit({

 windowMs: 15 * 60 * 1000,

 max: 100

});


app.use(limiter);
```

🌍 Real World Flow

Client → CORS → Helmet → Rate Limit → API → Database

🎯 Interview Questions

Q1: What is CORS?

👉 Allows frontend to access backend

Q2: What is Helmet?

👉 Secures HTTP headers

Q3: What is rate limiting?

👉 Controls number of requests

🎉 **CONGRATULATIONS**

[redacted]

✅ 100% BACKEND COMPLETED

🚀 **What You Know Now**

✓ Node.js
✓ Express
✓ MongoDB
✓ Mongoose
✓ CRUD
✓ Filters & Pagination
✓ Populate
✓ Aggregation
✓ File Upload
✓ Security

👉 This is **FULL MERN BACKEND (placement-ready)**

🚀 **NEXT BIG STEP**

Now you should:

1 Build Project (IMPORTANT)

👉 Secure File Storage System (your idea)

2 Then Deployment

👉 Make it LIVE

💡 **My Recommendation**

Next we do:

👉 **Build COMPLETE project step-by-step (industry level)**
👉 Then deploy it

Just say:

“start project” 🚀